

Prospector — Master Spec

One automated engine that sources, grounds, vets, ranks, packages and sells business opportunities as artifacts an entrepreneur (~16–60, any sector, any scale) will pay for. This document supersedes and consolidates the earlier drafts (system spec, core-engine spec, build brief, operating kit). It is the single source of truth.

Part 0 — What this is, and the one thing that matters

Product. A continuously refreshed catalogue of vetted business opportunities. Each entry (“Dossier”) carries a grounded thesis, sourced evidence, a build spec an LLM/dev can execute, a grounded go-to-market and ops plan, unit economics, and claim-checked marketing — composed into tiered packs and sold self-serve.

The thesis. When production is commoditised by AI, value moves to *judgment* (what’s worth building) and *grounding* (proof it’s real and current). Prospector industrialises both.

The one thing that matters. The product is **trust**, not idea-generation (which is free and saturated). Every listing is grounded in current, cited evidence and has survived a filter brutal enough to act on without redoing the homework. Two components carry the whole business — the **Verification Engine (Part 4)** and the **Kill-Filter (Part 4)**. Everything else is plumbing. If those two are mediocre, you’ve shipped a hallucinating grader. Spend your effort there.

Success criteria.

- Grounding integrity: ≥95% of published claims carry a resolvable source; **0** unverifiable quantitative claims.
- Filter discrimination: measured on the **fixed golden-set benchmark** (Part 13B), which must always show high discrimination — this, with the 100% grounding integrity above, is the anti-rubber-stamp guarantee. *Live* kill-rate is a different number: as generation learns to propose genuinely stronger ideas (Part 3), live survival should *rise* — that’s the goal, not rubber-stamping, **provided golden-set discrimination and grounding integrity hold**. Rising live pass-rate + stable benchmark discrimination = the engine working as designed.
- Freshness: 0 published Dossiers past their re-verification SLA.

- Unit economics: fully-loaded cost per published Dossier < target price ÷ 5.

Every one of these, and every feature below, is provable. Part 16 gives each feature an observable, automated pass condition wired into CI — nothing is “done” until its proof is green.

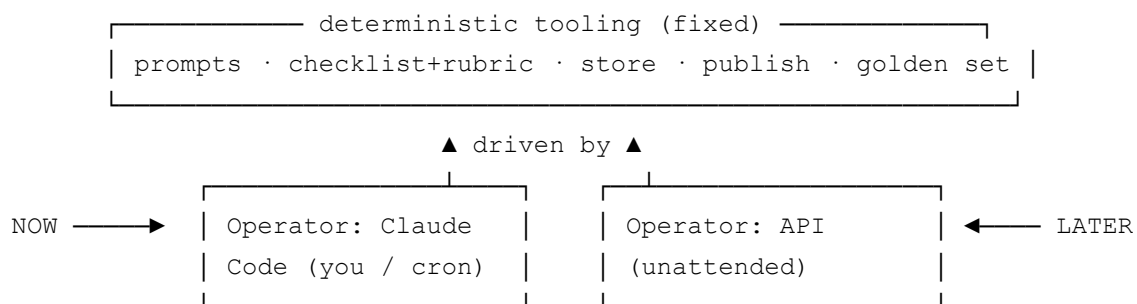
The open risk you own (Part 12): the engine produces excellent Dossiers; it does not make buyers show up. Distribution of the store is the real work and is decided before, not after, the build.

Part 1 — Operating model: short term vs long term

Short term — validate, ~£0. Claude Code is the *operator*: run on demand and on a schedule, against deterministic tooling, to generate → verify → produce content. Anything clearing the listing criteria is published. Cost: your existing Claude Code subscription + idle time. Paying per-token for an API while still proving the concept is wasteful.

Long term — scale. Swap the operator for an API-driven one: same tooling, unattended, higher volume, funded by revenue.

The rule that makes the split free of rework — the brain is pluggable:



Prompts, gates, golden set, publish step and tooling are operator-agnostic. **Bound (short term):** the binding limit is your Claude Code usage allowance — run supervised, modest batches inside it; producing your own work-product to sell is the intended use. When batches bump the cap, that’s the cue to fund the API operator. The supervised human role also serves as the cold-start quality check and liability backstop (Part 8).

Part 2 — The full automated pipeline

End to end, no human in the critical path (operator drives it):

```

[signals] → [generate + tag] → [dedup vs catalogue] → [pre-screen]
  → [VERIFY: 6 grounded checks, kill-fast] → [kill-filter] —kill→ (log dossie
    | pass
    ▼
[generate + GROUND: build spec, GTM, ops, financials]
  ▼
[generate + CLAIM-CHECK: marketing & listing content (4 types)]
  ▼
[compose PACKS: Scout / Operator / Founder-Investor (+ exclusivity / subscript
  ▼
[PUBLISH on pass: own store (Stripe) + syndicate (Gumroad)]
  ▼
[SCHEDULE revision: re-verify on SLA → refresh or delist]

```

For the long-term API version this is event-driven (a stage per worker, a queue between, KEDA-scaled, Terraform-provisioned). Short term, Claude Code executes the same stages as a procedure (Part 11). The logic is identical; only the operator differs.

Part 3 — Generation (unconstrained & divergent)

Principle — divergent generation, convergent verification. Generation is deliberately *unconstrained*: its one job is to produce many, varied, bold candidates. None of the filter's caution belongs here — nothing is killed, judged, hedged or rejected at generation time. All constraint lives downstream in verification (Part 4). A generator taught to self-censor produces timid, average ideas; keep it wild and let the filter do the killing.

Generation is normally fuelled by a **signal** (the richest, timeliest source of "why now") — any of these:

- demand / complaint signals (people loudly unhappy, paying for workarounds)
- cost shocks · technology shifts · behavioural/demographic change
- platform shifts (a new channel opening) · market gaps · a competitor failing
- regulatory / policy change (*one signal type among many*)

Each candidate annotates a *hypothesised* payer (a guess to be tested later, not judged here), ties its "why now" to the signal where it has one, and is tagged `sector / scale (side-hustle|smb|venture) / capital_required / skill_required / audience (b2b|b2c)` so operators of any kind or age can filter to their own. Range widely — adjacent sectors, unusual combinations, contrarian bets, and the occasional signal-free blue-sky pass are all encouraged. **Nothing is rejected at this stage**; weak monetisation, saturation, commodity

risk and legality are verification's call (the pre-screen below, then Part 4).

Prompt — generate (see Part 10).

Dedup. Embed-match each candidate against the existing catalogue; drop near-duplicates and mark the existing Dossier for refresh instead. Never list duplicates.

Pre-screen — the first (cheap) stage of *verification*, not a generation gate. Because all constraint lives in verification, the pre-screen is where the first kills happen: a fast, cheap pass that drops only the *obviously* dead — no acute pain, no plausibly solvent payer, plain undifferentiated commodity, or plainly illegal. **Creativity-preserving by design:** it is deliberately conservative and biased toward passing. An idea that is merely unusual, contrarian or hard-to-place is *not* obvious-dead — when in doubt, it passes through to full grounding (Part 4) rather than being killed cheaply. The pre-screen exists to save research budget on junk, never to suppress novelty; full verification, not triage, is what kills a non-obvious idea.

Adaptive creativity & generation optimisation

The core goal is **maximum yield of genuine survivors** — as many ideas as possible that *truly* pass the unchanged filter. Two levers drive it, and both make generation *smarter*, never the filter softer.

1. Better-aimed generation (unconstrained ≠ uninformed). Breadth and creativity are never reduced — but the generator is *informed* by what survives. It internalises the qualities that pass verification — acute pain, a solvent motivated payer, a durable/hard-to-commoditise core, a real distribution path, clean legality, and high automatability (Part 4) — as **design targets to aim toward**, not gates that prune. It still proposes wild, varied, contrarian ideas; it just aims them at real businesses rather than daydreams. Quality up, creativity untouched. (The killing still happens only in verification; generation never rejects.)

2. Failure-mode learning, fed forward as lateral guidance. Rejections are clustered by gate and reason (Part 8). The patterns ("recent consumer-app ideas keep dying on incumbency"; "rebate-style ideas keep failing value_durability") are fed *forward* to generation as guidance — never as bans. The instruction is never "don't generate X"; it is "this dead-end exists, for this reason — find the version that survives it." A killed territory stays open; only the failed *shape* is something to out-think.

Adaptive creativity controller — more creative the more it's rejected. A controller tracks the rolling kill-rate and *which gates* are doing the killing, and dials generation strategy in response:

- **Rejections rising → escalate exploration.** Raise diversity and switch on lateral lenses — analogical transfer from other domains, first-principles reframes, inverting the

problem, combining two signals, relaxing a shared assumption, crossing two unrelated sectors. Broaden the signal net. The harder it's getting, the further outside the box it goes.

- **A vein is passing → exploit it.** Narrow and deepen into the sector/shape that's clearing verification — more variations on what works.
- Two dials — **broaden ↔ narrow** and **conventional ↔ outside-the-box** — set by where survivors are actually coming from. An explore/exploit policy whose reward is *genuine* passes.

The anti-gaming guardrail (non-negotiable). "Optimise for passing" is safe *only* because the filter cannot be gamed into false passes: every verdict is grounded in external evidence the generator can't fabricate, and the golden set + cross-model checks (Parts 8–9, 16) catch any drift. So the only way to raise the pass-rate is to propose ideas that are *genuinely better* — which is exactly the goal. **Generation may use the pass/fail signal to get smarter; it may never cause verification to get softer.** The truth loop's veto (Part 8) is untouched and grounding integrity stays the hard invariant. If pass-rate ever rose while golden-set discrimination fell, that is the failure signal — and Part 16 tests for exactly that.

`generate.md` takes the controller's inputs: `{strategy_lens}`, `{exploration_level}`, `{target_qualities}`, and `{recent_failure_modes}` (shapes to out-think, never topics to avoid).

Part 4 — The core engine: verification, filter, ranking (the moat)

Six universal kill-checks

Applied to any business by the same bar. Each runs: generate disconfirming queries → retrieve → verdict.

check	question
<code>pain_reality</code>	Real, acute problem/desire — people suffering or paying to solve it?
<code>value_durability</code>	Is the value real and durable — not fabricated, already commoditised, or evaporating? (<i>a law change, fading demand, price collapse are special cases</i>)
<code>incumbency</code>	Does someone already solve this well (funded incumbent or dominant cheap option)?
	Does the payer have budget and motive (not a broke body, not a

payer_solvency	segment that won't pay)?
distribution	A low-friction route to the buyer (self-serve / forcing mechanism / existing channel)?
legality	Does the margin depend on breaking terms/law or falsifying a measurement?

Crisp where objective, hedged where inferential — by design. Checks against hard facts (a law, an incumbent, a published price) return sharp verdicts; fuzzy questions ("is consumer desire real?") ground on proxies (search trends, complaint volume, product traction) and return lower confidence. The confidence score + source-or-die rule make the engine say "plausible but unproven" rather than fake certainty.

The discipline (non-negotiable)

- **Source-or-die:** every factual/quantitative claim cites a retrievable source or is `unverifiable`. No unsourced numbers, ever.
- **Verdict-from-retrieval-only:** the model rules solely from passages it actually fetched. No prior knowledge. Silence $\Rightarrow$ `unverifiable`, **never** `supported`.
- **Adversarial pass:** after the checks, a red-team prompt argues the idea is dead from the gathered evidence; survivors must withstand it.

Prompts — query_gen, verdict, adversarial (Part 10).

Kill-Filter (hard gates, kill-fast)

Deterministic code over the verdicts. Evaluate cheapest decisive gates first; **stop at the first hard fail** (saves cost, keeps throughput flowing to contenders). `KILL` if any:

- `value_durability` $\in$ {`refuted`, `unverifiable`}
- `incumbency` $\neq$ `supported`
- `payer_solvency` $\in$ {`refuted`, `unverifiable`}
- `distribution` $\in$ {`refuted`, `unverifiable`}
- `legality` $\neq$ `refuted`
- `pain_reality` $\in$ {`refuted`, `unverifiable`}
- `adversarial.decisive` $\neq$ `true`
- any required verdict below the confidence floor

A KILL with a cited reason is a first-class output — render its dossier; it's the receipt that the filter is real. Target survival $\leq 10\%$ of verified, $\leq 1\%$ of raw.

Ranking (survivors only)

Score each axis 0–5; $\text{composite} = \Sigma(\text{score} \times \text{weight})$:

axis	weight
pain_acuity	0.20
money_provability	0.20
automatability	0.20
distribution	0.15
defensibility	0.15
build_feasibility	0.10

Automatability is a top-weighted axis — how much of the resulting business could run with little or no human labour. It is an *intrinsic property of the business model* (not operator-fit), scored realistically against what current, real tooling can actually do, never aspiration. The most fully-automatable ideas rank highest and price highest (Part 6): a business a buyer can run hands-off is the most valuable thing in the catalogue.

Fit-to-operator is still excluded from ranking (the store doesn't know the buyer); it's a browse filter via tags, never a gate. Don't confuse the two — automatability asks "can *this business* run itself?", operator-fit asks "does it suit *this buyer*?". Rank on the objective axes, automatability among them.

Prompt — score (Part 10).

Part 5 — Secondary artifacts (generated, then grounded or claim-checked)

Once an idea passes, produce what makes a pack worth paying for.

Grounded (premises must be sourced; you ground a plan's premises and kill its fantasy numbers — you cannot "verify" a strategy will work):

- **Build spec** — architecture + features an LLM/dev can execute; name only real, current,

maintained tools.

- **GTM plan** — channel, first-customer motion, wedge; every benchmark sourced (channel CAC, channel-audience fit, comparable motions, realistic conversion/pricing, marketing-legal constraints). Unsupported figures stamped `assumption - unverified`.
- **Ops plan** — cost assumptions (fulfilment, compliance, staffing, COGS) sourced against benchmarks.
- **Financial model** — inputs tied to the sourced benchmarks; no fabricated TAM; assumptions labelled.

Claim-checked (no new factual claims, no overstatement):

- **Marketing & listing content — all four types:** sales/listing-page copy, teaser + social snippets, SEO preview pages, launch/email copy.
- **Claim-consistency check** (its own stage): every factual statement in the copy must trace to a verified `claim` or sourced benchmark; nothing about a real named entity that isn't grounded; no overstatement. A piece that fails is regenerated, not published. *(You can't verify a tagline; you can forbid it from lying — which also caps false-advertising/defamation exposure.)*

Prompt — `claim_check` (Part 10).

Part 6 — Packs, tiers & sales surfaces

Packs are auto-composed views of one verified artifact graph — no human assembly.

Tier on depth and exclusivity:

Pack	Contains	Buyer
Scout	validated thesis + evidence table + score	"is this real?" — cheap proof
Operator	Scout + build spec + grounded GTM + unit economics + ops plan	someone about to build
Founder / Investor	Operator + competitive teardown + grounded financial model + risk register + market memo + LLM-ready build spec	serious operator / capital

Orthogonal levers:

- **Pricing — automatability commands the premium.** Listing price tracks the composite score, with **automatability weighted hardest**: the strongest, most fully-automatable opportunities get the top tier and the highest price (and are the natural exclusivity picks), because an idea a buyer can run with little or no human labour is the catalogue's most valuable asset.
- **Exclusivity** — buy it solo; delisted after $\leq N$ sales (scarcity premium).
- **Subscription** — N fresh validated theses / month (pairs with the decay loop; selling freshness).
- **Investor reframe** — the same output recomposed as deal-flow/market-intelligence (thesis + emerging players + the driving shift), not a build kit. One product, two markets.

Both sales surfaces:

- **Own storefront (Stripe)** — canonical catalogue, all tiers + exclusivity + subscription, the trust surface ("verified {date}", source count, re-verify SLA). Auto-publish on pass.
- **Syndication (Gumroad / Lemon Squeezy)** — push the entry tiers (Scout/Operator) via API for reach; canonical fulfilment + premium/exclusive tiers stay on your store. A funnel, not a second source of truth. Both free to start (a % on sale, nothing upfront).

Listing/marketing copy comes from Part 5's claim-checked artifacts.

Part 7 — Dedup & scheduled revision (the decay loop)

Ideas rot: markets move, competitors ship. Each published Dossier carries `reverify_due_at` (SLA by volatility — fast-moving ~30d, slow ~90d). A scheduled pass re-runs verification on due Dossiers; if a hard gate now trips, mark `stale` → auto-delist (optionally notify subscribers); re-confirmed ones get a fresh date. This protects trust and justifies recurring revenue (you sell freshness, not a static list). Dedup (Part 3) prevents duplicates entering; the decay loop removes ones that have died.

Part 8 — Autonomy & continuous learning

Posture. Operationally autonomous; the operator runs unattended (long term) or in supervised batches (short term). Learning and self-adjustment come from data. Human role decays toward near-zero as data accrues — full hands-off is a convergence target, not a Day-0 switch.

The cardinal rule: two loops, never merged.

Loop	Measures	Optimises	Live from
Demand	desirability — what buyers want	conversion, pricing, framing, sector mix, generation targeting	first sale
Truth	accuracy — whether claims are real	grounding integrity; gate/threshold calibration	Day 0 (internal)

Sales measures desirability, **not** truth. Optimise on sales alone and the engine drifts toward confident, exciting, plausible output — hallucination with better lighting — because that converts. For a product whose only moat is being right, that's the one failure that ends it. **The demand loop tunes what to offer; the truth loop governs what may ship, and holds a veto the demand loop never gets.**

Truth loop (replaces standing human QA): deterministic source-resolution (fetch each cited URL, confirm the passage is there), cross-model verification (a second pass checks verdicts against the same sources), golden-set regression (Part 13B — halts any change that worsens scores). Internal pre-screen calibration runs Day 0 with no external data.

Learning from rejections — without hurting performance or creativity. Every KILL is stored with its reason, gate, evidence and date (Part 9 audit). The system mines that history under three rules:

- *It speeds verification; it never gates generation.* A new candidate materially identical to a previously-killed one can be fast-pathed to the same KILL — saving redundant grounding. That's a throughput win, applied in verification, never a filter on what may be generated.
- *Never a denylist; markets change.* A past rejection is never a permanent ban on an idea or a class of ideas, and generation is never handed a "don't generate X" list. Rejection memory may at most pass *soft, advisory* context to verification ("this exact dead-end has been seen, with this evidence"); it can never suppress an idea class or make the generator timid. A once-dead idea is free to be re-proposed and, if the world has moved, to pass this time.
- *Aggregate to calibrate the filter, not to bias the output.* Rejection patterns feed the **truth loop** — which gates fire most, where false-kills/false-passes cluster (caught by golden-set + cross-model checks), whether thresholds need tuning. This sharpens accuracy. It is never wired into the demand loop and never used to chase "ideas that pass more easily."

Three guardrails keep this from degrading anything:

1. *Freshness-gated* — a fast-path kill is trusted only while its stored evidence is still in-SLA (Part 7); otherwise the idea gets full re-verification (a stale kill is re-opened, not reused).
2. *Confidence fallback* — anything short of a near-identical match falls back to full verification. The fast-path only ever saves work on certainties; it never emits a kill it isn't sure of, so it can't trade accuracy for speed.
3. *Regression-protected* — any change to rejection-learning runs against the golden set and is reverted if discrimination worsens. Both creativity (generation breadth) and accuracy (verification) are measured and regress-tested (Part 16).

The generation-yield loop (Part 3). The same rejection signal also drives the adaptive creativity controller: as kill-rate rises, generation gets *more* creative and better-aimed, to lift the yield of genuine survivors. This is a **third, generation-side loop**, distinct from demand and truth. It consumes verification's pass/fail signal but — by the anti-gaming guardrail — can only make generation smarter, never the filter softer. Demand never touches truth; truth keeps its veto; this loop sits upstream of both and is bound by the same rule: a rising pass-rate is only legitimate while golden-set discrimination and grounding integrity hold.

Two honest limits that remain:

1. **Cold-start** has no signal to learn from until the market responds. Pre-seed the golden set; run light human sampling that decays to near-zero. (Short term, the supervised operator *is* this.)
2. **Liability is not a loop.** A wrong claim about a real entity, or a Dossier a buyer loses money on, lands on you. Retain sources + model versions for dispute; gate claims naming real entities behind the strictest source-resolution; carry it as an operating risk.

Part 9 — Production-hardening

- **Strict structured output + repair-retries** on every model call; a bad parse never crashes a run.
- **Graceful degradation** — a failed search/fetch downgrades that check to `unverifiable`, never kills the idea/batch; backoff + rate-limit handling.
- **Kill-fast short-circuit** — cheapest decisive gates first; stop at first hard fail.
- **Bounded concurrency** — parallel checks per idea, capped parallel ideas, rate-limit-aware.
- **Three-layer caching** — search results, fetched pages, (idea, check) verdicts.

- **Golden-set as CI** — runs on every prompt/config change; blocks regressions.
 - **Pinned model + full audit** — prompts, model version, sources, verdicts persisted; every artifact reproducible and disputable.
 - **Spend guards** — daily caps with circuit-breakers; per-run cost-per-passed-idea printed.
-

Part 10 — Prompts (the IP, verbatim)

All return strict JSON. {...} injected. One model throughout.

generate.md

SYSTEM: You are a bold, divergent idea generator surfacing monetisable business opportunities across ANY sector, scale, audience and background. Your job is breadth and originality: produce many varied, ambitious, sometimes contrarian ideas. Do NOT self-censor, judge, hedge or reject — a separate verification stage does all the killing. Aim ideas at real businesses (see target_qualities) while staying maximally creative; quality and creativity are not in tension. The higher the exploration_level the further outside the box you go (analogical transfer, first-principles reframing, inverting the problem, combining signals, crossing unrelated sectors).

USER: Signal (optional): {signal_text} Sector hint (optional): {sector}

Strategy lens: {strategy_lens} Exploration level 0-1: {exploration_level}

Target qualities to aim for (NOT gates): {target_qualities}

Recent failure modes to OUT-THINK (never avoid the topic — beat the shape): {recent_failures}

Produce up to {k} DISTINCT opportunities. Range widely; if no signal, generate broadly. Each names a *hypothesised* payer (a guess to be tested, not "consumers" in general). "why_now" references the signal where there is one. Tag each: sector, scale, capital_required, skill_required, audience, and automatability (how much of the business could run with little/no human labour). If monetisation is only generic or a no-named-payer subscription, FLAG ("weak_monetisation": true) — do not drop it. Output ONLY a JSON array of {title, one_liner, hypothesis, who_pays, why_now, tag

prescreen.md

SYSTEM: You are the first cheap triage inside verification. Kill ONLY the obvious dead; when in doubt, PASS. You protect research budget WITHOUT suppressing novelty. An unusual, contrarian or hard-to-place idea is NOT obvious-dead. Same bar, any signal

USER: Candidate: {candidate_json}

Set keep=false ONLY if clearly true: no conceivable acute pain/desire; no plausible solvent payer at all; plain undifferentiated commodity with no angle; or plainly illegal / dependent on breaking terms or falsifying data. Anything genuinely uncertain → keep=true (full grounding will judge it).

Output ONLY: {"keep": bool, "reason": "<one sentence>"}

query_gen.md

SYSTEM: You write web search queries that would EXPOSE a business idea as dead.

USER: Candidate: {candidate_json} Check — {check_name}: {check_question}

Write 1-3 queries most likely to surface DISCONFIRMING evidence (named entities/products/laws, "reform/abolished/banned/discontinued", "alternatives/competitors", "market size/demand", current year).

Output ONLY a JSON array of query strings.

verdict.md (the moat)

SYSTEM: You are a ruthless, evidence-bound analyst. Rule ONLY from the passages provided. No prior knowledge. If the passages don't address the question, verdict is "unverifiable". NEVER "supported" without a passage that directly supports it. Cite the source_ids you relied on. Confident wrongness is the worst outcome.

USER: Candidate: {candidate_json} Check — {check_name}: {check_question}

Passages: {for each: [source_id] (url, published_at) text}

Output ONLY: {"verdict":"supported|refuted|unverifiable","confidence":0.0, "rationale":"<=2 sentences, grounded strictly in cited passages","citations":["s

adversarial.md

SYSTEM: Your only job is to kill this idea using the evidence gathered.

USER: Candidate: {candidate_json} All claims + passages: {verification_json}

Make the strongest EVIDENCE-BASED case it's dead or not worth building. Cite source_ids. State whether the case is decisive.

Output ONLY: {"kill_case":"","decisive": bool, "citations":[...]}

score.md

SYSTEM: Score a vetted opportunity on six axes, 0-5, grounded ONLY in the provide claims. Same standard for any sector. Score `automatability` REALISTICALLY against current, real tooling can actually do today — not aspiration. Justify each in one citing source_ids where used.

USER: Candidate: {candidate_json} Claims: {claims_json}

Axes: pain_acuity, money_provability, distribution, defensibility, build_feasibil

Output ONLY: {"scores":{"axis:int...}, "justification":{"axis":"..."}}

content_gen.md (writes the listing/marketing copy in the brand voice — full spec Part 15A)

SYSTEM: You write listing and marketing copy for a vetted business opportunity, i

our brand voice. The voice: clear, inclusive, focused on the reader; make complex things simple. Plain English, no jargon, no hype. Active voice. Lead with what it is and how it helps the reader BEFORE the reasoning. Confident because the claims are grounded – never rowdy or overbearing. Warm, like a knowledgeable friend, never flippant. Light wit is fine; starting a sentence with "And" or "But" is fine. Write for anyone, any sector, any age, any background – no gendered, age-coded or insider assumptions; if a term needs specialist knowledge, explain it plainly. HARD RULE: state ONLY what the provided verified claims support. No new facts, no overstatement – the voice never overrides the evidence. (A separate claim-check will reject anything that strays.)

USER: Opportunity: {candidate_json} Verified claims/benchmarks: {claims_json}
Type: {one of: listing_page | teaser_social | seo_preview | launch_email}
Output ONLY: {"type":"...", "copy":"..."}

claim_check.md

SYSTEM: You check marketing/listing copy for claim-consistency. Every factual statement must trace to a provided verified Claim or sourced benchmark. Flag any new unsourced claim, any overstatement beyond the evidence, any unverified mention of a real named entity.

USER: Copy: {copy} Verified claims/benchmarks: {claims_json}
Output ONLY: {"pass": bool, "violations":[{"text":"","issue":""}]}

Part 11 — Repo layout & operating procedure

```
prospector/  
  CLAUDE.md          # operating rules (below)  
  RUN.md             # per-run procedure (below)  
  config.yaml        # Part 13A  
  prompts/           # the prompt files from Part 10 (eight)  
  fixtures/golden_set.json # Part 13B  
  store/             # local run + catalogue state  
  publish/publish.py  # push a PASS to own store + syndicate (build first)  
  signals/           # pasted signals, one per file
```

CLAUDE.md (root): the cardinal rules — source-or-die; verdict-from-retrieval-only; the filter is universal; kill-fast; a KILL with a cited reason is first-class; publish only passes; follow RUN.md; use web tools for grounding; write every run to store; run supervised batches inside the usage allowance; no hosted service / no API-key calls / no infra beyond this repo.

RUN.md (per run):

1. (*discover*) GENERATE from the signal (generate.md), capped.
2. DEDUP vs catalogue; drop near-dupes; mark existing for refresh.
3. PRE-SCREEN (prescreen.md); kill obvious losers.
4. VERIFY (kill-fast): per check, query_gen.md → fetch real pages → verdict.md; stop at first hard fail; then adversarial.md.
5. GATE (config) → KILL or PASS; render a dossier either way to store/.
6. (*PASS*) grounded secondary artifacts (score.md) + claim-checked content (claim_check.md); regenerate anything failing the claim-check.
7. (*PASS*) run publish/publish.py.
8. Print run summary: decision, gate fired, sources, estimated cost.

Schedule + on-demand. On demand: run RUN.md against a signal or a single `vet`. On schedule: a cron job invokes the Claude Code CLI against RUN.md for the day's signal(s); keep `batch_size` modest to sit inside the usage allowance.

publish/publish.py (build first): on a PASS, upsert the listing on your own store (static site + Stripe) — all tiers + exclusivity + subscription — and syndicate the entry tiers to Gumroad via API. Listing copy = the claim-checked marketing artifacts.

Part 12 — The open risk: distribution of the store

The engine produces excellent, trustworthy Dossiers; it does **not** make buyers show up. That, not the build, decides whether this earns. Decide the channel before you scale. Options:

- **A free “kill my idea” front door** — users submit their own idea, you run the Verification Engine on it free, upsell the catalogue. Sidesteps the “selling strangers’ ideas” problem and attacks the market’s complaint (vague, untrustworthy scores) with grounded, sourced verdicts. (*Strongest option.*)
 - **Content/SEO off the by-product** — the redacted teaser theses are sector-trend content; a built-in top of funnel if you can rank it.
 - **Audience-first** — build a following around the rigour (the kills, the receipts) and sell into it.
-

Part 13 — Reference

13A — config.yaml

```
operator: claude_code          # swap to "api" later; tooling unchanged
model: claude
retrieval: { queries_per_check: 3, results_per_query: 4, max_passage_chars: 1500
thresholds: { confidence_floor: 0.6, min_composite_to_pass: 3.2 }
hard_gates:
  - value_durability: [refuted, unverifiable]
  - incumbency: [supported]
  - payer_solvency: [refuted, unverifiable]
  - distribution: [refuted, unverifiable]
  - legality: [refuted]
  - pain_reality: [refuted, unverifiable]
  - adversarial_decisive: true
weights: { pain_acuity: 0.20, money_provability: 0.20, automatability: 0.20, dist
generation:
  candidates_per_signal: 20
  controller:                # adaptive creativity — Part 3
    killrate_window: 50      # rolling sample
    exploration_min: 0.2
    exploration_max: 0.9      # exploration rises as kill-rate rises
    lenses: [broaden, narrow, analogical, first_principles, invert, combine_signa
    target_qualities: [acute_pain, solvent_motivated_payer, durable_hard_core, re
listing:
  publish_on: pass
  surfaces: [own_store, syndicate]
  tiers: [scout, operator, founder_investor]
  exclusivity: true
  subscription: true
  pricing: { driver: composite, premium_axis: automatability } # price tracks s
schedule: { cadence: daily, batch_size: 5 }
```

13B — fixtures/golden_set.json (acceptance test; expand over time)

Mixed sectors; the engine must produce the right decision **for the right, cited reason**.

idea	expected	gate	must surface
Haulage HMRC fuel-duty PTO rebate	kill	value_durability	2022 red- diesel reform removed off- road

			entitlement for haulage
Construction retention recovery	kill	value_durability/distribution	retentions being banned; loss is insolvency-driven
AI subscription spend-audit bot	kill	incumbency	funded incumbents already ship SaaS-spend management
Care-home council fee arbitrage	kill	payer_solvency	councils chronically under-funded; payer can't pay
Token/credit resale clearinghouse	kill	legality	resale/account-sharing violates provider terms
Generic AI meal-planner app	kill	incumbency/pain_reality	saturated consumer category, no acute pain (non-reg test)
Unified-API niche bridge (open-core)	pass	—	proven pattern; moat = un-shortcuttable integration work
Construction smash-and-grab (invalid Pay Less Notice)	pass	—	solvent payer + statutory adjudication forcing mechanism

(JSON form provided as a standalone `golden_set.json` for the harness.)

Part 14 — Build roadmap

Phase 0 is a **quality experiment**, not a sprint. Prove the moat before building the rest.

1. **Wrappers/tooling** — repo, config loader, store, the publish stub; for the Claude Code operator, the prompts + RUN.md + CLAUDE.md.
2. **One check, end to end** — wire `value_durability` on the fuel-duty idea. **Pass condition: returns `refuted`, citing the April 2022 red-diesel reform, with a real URL.** If it grounds, the moat is real; if not, fix prompts/retrieval before anything else.
3. **Full vet path** — all six checks + adversarial + hard gates + scoring; render PASS/KILL dossiers.
4. **Golden-set harness** — run 13B; iterate prompts/thresholds until it discriminates across mixed sectors. This is the Phase-0 acceptance gate.
5. **Discover path** — signal intake + generation + pre-screen + tagging.
6. **Secondary artifacts + claim-checked content + packs** (Parts 5–6).
7. **Publish-on-pass** to both surfaces (Part 11).
8. **Decay loop + the demand / truth / generation-yield loops, including the adaptive-creativity controller** (Parts 3, 7–8); harden (Part 9).
9. **Graduate** to the API operator when usage outgrows the subscription and revenue justifies it.

Build 1–4 first. If the survivors hold up to your own scrutiny with real sources and the dead ideas die for the right cited reasons, you have the moat — and a reason to build everything above it. If not, you’ve learned it cheaply.

Part 15 — Voice, inclusivity & app integration

This part governs *how the artifacts speak, who they’re for, and how the engine plugs into your own web and mobile apps*. It overlays the pipeline without touching the verification core: the moat is still grounding + the filter, and the claim-check (Part 5) always outranks voice.

15A — Brand voice (Monzo-style)

Everything a buyer reads — listing pages, teasers, SEO previews, emails — adopts a Monzo-style voice. Monzo publishes its guide at monzo.com/tone-of-voice; we borrow its spine.

The engine applies this via `content_gen.md` (Part 10).

The always-on thread — “Straightforward Kindness”: clear, inclusive, focused on the reader; go out of the way to make complex things simple. Every word, all the time.

Concrete rules baked into the content prompt:

- **Reader first.** Lead with what it is and how it affects the reader; explain the reasoning *after* the impact, not before.
- **Plain English, no jargon.** Break down anything specialist without talking down. A term that needs insider knowledge gets explained.
- **Active voice.** “You recover the withheld cash,” not “the cash may be recovered.”
- **Transparent.** Say what a thing is and why, plainly; no ambiguity, no hedging filler.
- **Confident, not overbearing.** The claims are grounded, so the copy is calm and certain — never rowdy, never hype.
- **Warm, not flippant.** Like a knowledgeable friend telling you something useful — not a corporation, not a comedian. Light wit is welcome; opening a sentence with “And” or “But” is fine.
- **Everyday magic, sparingly.** A moment of warmth or a sharp phrase belongs in brand/marketing copy — never at the cost of clarity or accuracy.

The one hard boundary — honesty outranks voice. This product’s moat is being right. The voice makes a grounded claim *land*; it never adds, inflates, or softens a claim beyond the evidence. `content_gen.md` may use only verified claims; `claim_check.md` runs after and rejects anything that strays, however nicely it’s phrased. Warmth and rigour don’t conflict here — plain, honest, reader-first writing is exactly what a trust product should sound like.

(Monzo’s guide permits a few friendly emojis; given the register and your preference, the artifacts stay emoji-free. Keep the plainness and warmth, drop the emoji.)

15B — Inclusive, wide curation (all sectors, ages, genders, backgrounds)

The catalogue is for everyone, and so is the language. Two commitments:

Breadth of curation. Generation spans sectors, scales (side-hustle → venture), capital and skill levels, and audiences (B2C/B2B) — already enforced by the sector-agnostic signal taxonomy and tagging (Part 3). Make breadth a curation KPI: no single sector, scale or operator-type should dominate the live catalogue. Anyone who lands finds opportunities that fit, and filters to them by tag.

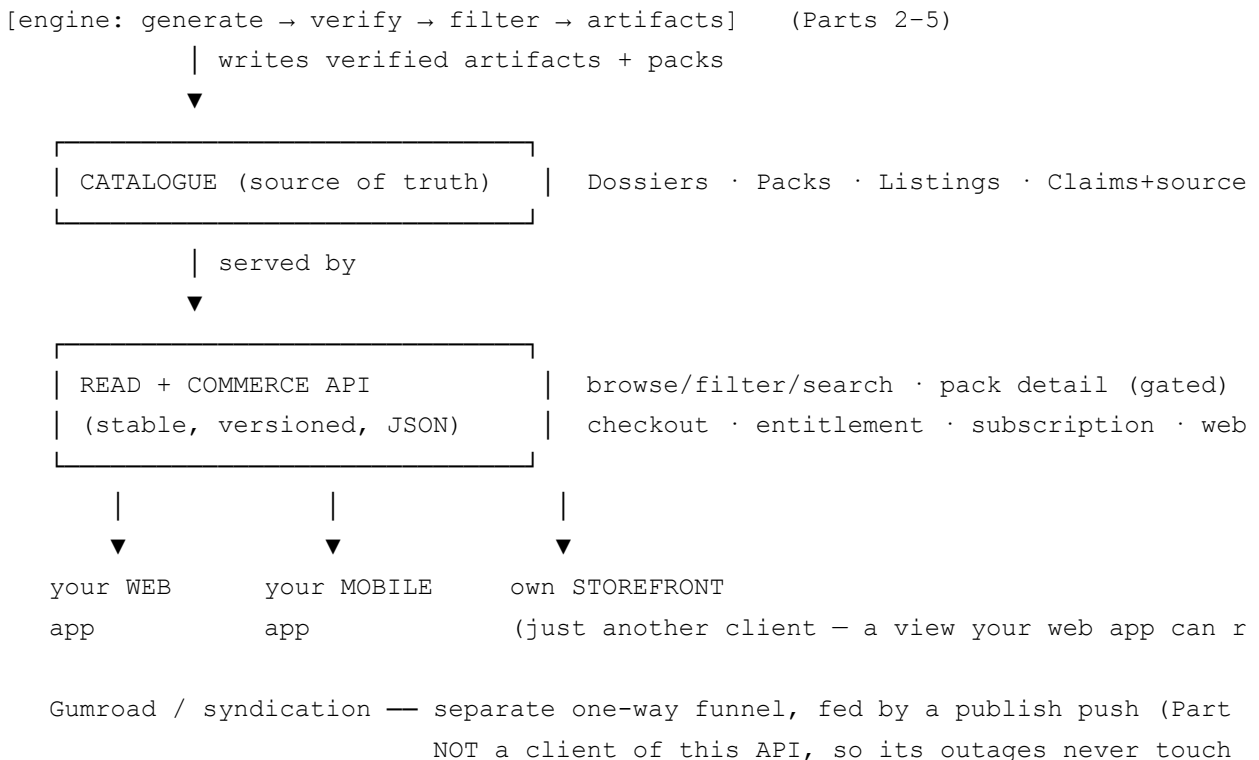
Inclusive language — the same thread as the voice (Monzo's principle: open, inclusive and welcoming to everyone). The artifacts:

- assume no gender, age, culture or insider background of reader or operator;
- avoid demographic coding — an idea isn't "for young men" or "for mums," it's described by what it does and who pays;
- read clearly for a sharp 16-year-old and a 60-year-old career-changer alike — plain words, explained terms, accessible reading level;
- describe the payer and the work, never a stereotype of who'd run it.

This widens the addressable buyer base, costs nothing, and removes a whole class of ways to lose a reader.

15C — Headless core + API (built to feed your web & mobile apps)

Build Prospector as a **headless core with an API**, not a standalone storefront. The engine and catalogue are the product; the storefront, your web app and your mobile app are all *clients* of one API. This is the separation that lets it drop into your existing apps with no retrofit.



What this means for the build:

- **API-first.** The data contracts / schemas (Part 13) *are* the API resources — Dossier ,

`Pack`, `Listing`, `Claim` (with sources), `Entitlement`. Design a stable, versioned read API plus commerce endpoints (checkout, entitlement check, subscription status, webhooks).

- **Entitlement, not bespoke store logic.** A Stripe purchase grants an entitlement; any client — web, mobile, storefront — unlocks gated pack content by checking entitlement against the same API. Teasers public; full Dossiers gated.
- **The storefront is a thin client.** Part 6's "own storefront (Stripe)" is one renderer of this API, and the same surface your web app can embed. The mobile app hits identical endpoints. One source of truth, three faces.
- **Syndication stays separate.** Gumroad/Lemon Squeezy remain a one-way publish target for the entry tiers (a funnel), never a consumer of your API.
- **Shared auth.** Reuse your apps' existing identity/session so a user's purchases and subscription carry across web, mobile and storefront.

Roadmap impact: at Part 14 **step 7 (publish-on-pass)**, build the catalogue + read/commerce API *first* and make the storefront the first client of it — instead of a bespoke store you'd later retrofit for the apps. The engine (steps 1–6) is unchanged; this only shapes how its output is served.

Part 16 — Proof of function (acceptance & test plan)

Every feature has an observable, automated pass condition. Nothing is "done" until its proof is green and in CI. Proof comes in three layers: (1) the **golden set** (Part 13B — truth & discrimination), (2) a **per-feature test suite** (behaviour, units, invariants, fault-injection), and (3) **live metrics** (the running system stays inside the Part 0 success criteria). The suite runs on every change; a red proof blocks ship.

A note on honesty: most features get a hard, deterministic proof. Two — *voice* and *inclusive language* — are partly subjective, so they're proven against **measurable proxies** (lints) plus a **sampled rubric grade** from a separate grader prompt. That proves adherence to defined, checkable traits, not "good writing" in the abstract — and it's stated as such rather than overclaimed.

Feature → proof map

Generation (unconstrained)

Feature	"Works" means	Proof

Divergent generation	One signal yields $\geq k$ distinct candidates across $\geq N$ distinct sectors/tags; nothing judged or dropped here	Run on a fixture signal; assert count, tag-diversity metric, and that <code>weak_monetisation</code> items are <i>flagged not absent</i> (behavioural)
No creativity suppression	Rejection history never blocks an idea class	Seed memory with a killed class, generate; assert ideas in that class still appear (invariant)
Adaptive creativity	Exploration escalates as rejections rise	Simulate rising kill-rate; assert <code>exploration_level</code> rises and the strategy lens varies (controller test)
Failure-learning never bans	A failed shape raises out-thinking, never blocks the topic	Extend the no-suppression invariant: a prior-failed topic still generates, and a stronger variant can PASS

Triage & dedup | Dedup | Catches a true near-duplicate, leaves genuinely distinct ideas alone | Positive + negative fixture pair (behavioural) | | Pre-screen kills obvious-dead | Plainly-illegal / no-payer / pure-commodity $\rightarrow$ `keep=false` | Fixture set (behavioural) | | **Pre-screen preserves novelty** | An unusual-but-viable / contrarian idea $\rightarrow$ `keep=true` (passes to full grounding) | Novelty fixtures must survive triage (invariant) — the key creativity proof |

The moat — six checks, verdict discipline, adversarial | Each check correct | Right verdict for the right cited reason | Golden set per-check {`verdict`, `gate`, `must-cite domain`} (Part 13B) | | **Source-or-die** | No supporting passages $\Rightarrow$ `unverifiable`, never supported; no unsourced number ever ships | “No-evidence $\Rightarrow$ unverifiable” fixture + output scan that fails on any number lacking a citation | | **Verdict-from-retrieval-only** | Verdict follows the passages, not the model’s prior | Fixture where passages refute/omit a “known” fact; assert verdict tracks passages | | Flagship acceptance | Fuel-duty `value_durability` $\rightarrow$ `refuted`, citing 2022 red-diesel reform, **resolvable URL** | Golden fixture with live source-resolution (Part 14 step 2) | | Adversarial | Decisive kill on a known-dead idea, with citations | Golden dead-idea fixture: assert `decisive=true` + citations |

Filter, scoring, artifacts, content | Kill-filter gates | Each hard-gate condition $\Rightarrow$ KILL; clean set $\Rightarrow$ PASS | Pure unit tests over synthetic verdict sets (no model) | | Kill-fast | First hard-fail stops further checks | Instrumented call-count / short-circuit assertion | | Scoring maths | Composite computed exactly from weights; clearly-stronger idea outranks weaker | Unit (exact weight maths) + ordering fixture | | Fit-to-operator excluded from rank | Changing operator-fit tags never changes rank | Invariant test | | Automatability ranks & prices higher | The most-automatable passing idea wins both rank and price | Fixture: two passing ideas equal on every other axis — assert the automatable one ranks higher and prices higher | | Automatability realistic | “Fully automatable” beyond current tooling is scored down | Fixture

with an unsupported automation claim → low automatability score | | Artifact grounding | A planted fantasy number (e.g. invented TAM) is labelled `assumption – unverified` or stripped; cited benchmarks resolve | Fixture with planted figure | | **Claim-check (honesty > voice)** | Copy with an unsupported claim ⇒ `pass=false` + violation; clean copy ⇒ `pass=true` | Positive + negative fixtures + end-to-end (content_gen → claim_check): no published copy carries an unsupported claim | | Voice (Monzo-style) | Measurable traits met | Deterministic voice-lint (reading level in range, passive-voice ratio < threshold, banned jargon/hype absent, no emoji) + sampled rubric grade ≥ threshold | | Inclusive language | No demographic-coded targeting | Demographic-coding lint + sampled rubric → zero coded targeting | | Breadth KPI | Catalogue stays varied | Live metric: no single sector/scale/operator-type exceeds its cap of the live catalogue |

Packs, publish, API | Pack composition | Each tier contains exactly its specified artifacts; gated content absent from teaser tiers; exclusivity delists after N sales | Unit + sales simulation | | **Publish only on PASS** | A KILL never publishes; listing carries trust metadata (verified date, source count, SLA) | Pipeline test: assert publish called iff PASS + metadata present | | Both surfaces resilient | Own-store upsert + syndication push both fire for entry tiers; a syndication outage never blocks canonical publish | Integration test with stubbed syndication failure | | **API entitlement gating** | Un-entitled request for full Dossier refused; teaser public; valid entitlement unlocks | API auth tests (positive + negative) | | API contract stable | Web/mobile clients won't break on a change | Versioned-schema contract tests on each resource |

Loops, learning, operator, hardening | Decay loop | Past-SLA Dossier is re-verified; now-failing ⇒ stale + delisted; still-valid ⇒ date refreshed | Clock time-travel simulation; live metric: 0 live Dossiers past SLA | | Rejection fast-path | Near-identical killed idea (evidence in-SLA) reuses the KILL with fewer calls | Instrumented simulation | | Fast-path freshness gate | Same idea, evidence past SLA ⇒ full re-verification (not reused) | Simulation | | Confidence fallback | Loosely-similar (not near-identical) ⇒ full verification, no fast-path kill | Fixture | | **Two loops never merge** | Sales/demand data never changes a truth verdict | Invariant: feed high sales signal on weak-evidence idea; assert verdict/gate unchanged — the single most important guardrail proof | | **Yield without gaming** | Survivor yield rises while the filter stays exactly as hard | Representative run with the controller on: assert a rising live pass-rate never coincides with golden-set discrimination falling or grounding integrity dropping — the anti-gaming proof | | Operator-pluggable | Identical decisions under the Claude Code operator and the API operator | Run the core suite against both adapters; assert identical fixture decisions | | Graceful degradation | A failed search/fetch downgrades that check to `unverifiable` ; run completes | Fault-injection test | | Structured-output repair | Malformed model output is repaired or fails safe; no crash | Fault-injection test | | Spend guard | Cost over the daily cap trips the circuit-breaker | Simulation |

Global (live release gates — Part 0): grounding integrity ≥95% sourced · 0 unverifiable

quantitative claims published · **high golden-set discrimination** (a *rising* live pass-rate is fine, and expected, while this holds) · **0** stale published Dossiers · fully-loaded cost/Dossier < price ÷ 5. Run continuously as dashboards with alerts, and as gates on each representative batch.

Harness layout & CI

```
tests/
  unit/          # gate logic, scoring maths, pack composition (no model)
  behavioural/   # generation diversity, dedup, pre-screen, claim-check, publish
  invariants/    # no-suppression, two-loops-never-merge, fit-excluded-from-rank,
  integration/   # API auth/entitlement, contract, both-surfaces, syndication-outa
  faults/        # degradation, output-repair, spend-guard
  sim/           # decay clock, rejection fast-path freshness
graders/         # voice + inclusivity rubric prompts (sampled)
lints/           # voice-lint, demographic-coding lint, unsourced-number scan
fixtures/        # golden_set.json + per-feature fixtures
metrics/         # live KPI checks wired to the Part 0 criteria
```

CI gates: unit + behavioural + invariants + integration + faults run on **every** change; the **golden set runs on every prompt/config change** (Part 9); rubric graders run nightly on sampled output; live metrics run continuously with alerting. Any red proof blocks ship.

Definition of done

A feature is done only when: its code is merged, **its proof (test / fixture / metric) is green in CI**, and its audit trail (prompts, model version, sources, verdicts) is captured. A feature without a passing proof is **not** done.

This maps directly onto the Part 14 roadmap — each step ships with the proofs for what it introduces: step 2 ships the fuel-duty source-resolution proof; step 3 the gate + verdict-discipline proofs; step 4 the full golden set; steps 5–8 their generation-diversity, claim-check/voice/inclusivity, publish/API-entitlement, and decay/learning/two-loops proofs. By the time the roadmap is complete, every feature in this document has a green proof behind it.